Detecting AI-Generated Text with Invariant Representations of Probability Distributions

Zhuoqin Wang
Department of Computer Science
University of Chicago
Chicago, IL 60637
zjw2005@uchicago.edu

Abstract

We study the problem of detecting AI-generated text under realistic conditions where the generating distribution is unknown and may differ from available reference models due to model mismatch or fine-tuning. Existing probabilistic detectors, such as DetectGPT, rely on likelihood-based signals computed under a fixed reference model, and their effectiveness can degrade under distributional misalignment.

We propose an invariant probabilistic detection framework that extracts token-level probability features—including normalized log-likelihood, token rank, entropy, and perturbation-based curvature—from a diverse set of reference language models. These features are normalized within each model and aggregated to capture cross-model agreement and dynamics, reducing dependence on any single assumed distribution. We then learn a representation over these features using adversarial training to remove model and domain information, ensuring that the detector focuses on generation-process signals rather than stylistic cues.

1 Introduction

As AI-generated text becomes increasingly prevalent, reliable detection under realistic conditions is critical. Stylometric detectors such as GPTZero rely on surface-level statistical patterns that break under domain shift and are easily gamed. Probabilistic detectors like DetectGPT improve on this by exploiting the observation that AI-generated text tends to sit near a local maximum of the likelihood landscape — small perturbations lower the likelihood more sharply for AI text than for human text. However, these methods are gated by a single reference model: when the generating model differs significantly from the reference, the curvature signal washes out and produces high false-negative rates.

We propose combining probabilistic detection signals grounded in the generation process with invariant representation learning to remove generator and domain bias. Central to our approach is a formal treatment of what it means for a detector to be invariant: we define the space of AI-generating distributions as $\mathcal{P}_{\text{AI}} = \{p(x \mid M, c) : M \in \mathcal{M},\, c \in \mathcal{C}\}$, parameterized by generating model M and conditioning context c. A robust detector must generalize across this entire family rather than a single point in it. In practice we approximate coverage of this space with a discrete set of reference models and text domains, and we discuss how techniques such as activation steering vectors could enable denser, more principled coverage in future work. Our contributions are:

- **Formal definition of the invariant detection problem.** We define the family of AI-generating distributions $\mathcal{P}_{\text{AI}}$ and frame detection as the problem of building a classifier invariant across all distributions in this family, motivating both our feature design and our training objectives.

Preprint.

- **Invariant probabilistic features and detection.** We extract token-level probabilistic signals — log-likelihood, rank, margin, entropy, and curvature — from multiple reference models, and normalize them to isolate features that reflect the AI generation process rather than any particular model's distribution. We then train a detector to recognize these invariant features using a combined loss of classification, adversarial gradient reversal on generator and domain heads, and contrastive learning, ensuring the learned representation captures generation-specific patterns and not stylistic or model-correlated artifacts.
- **Empirical validation on unseen generators.**

2 Defining Invariant Domains for AI-Generated Text Detection

A detector that generalizes reliably must be invariant not to a single generating distribution, but to an entire *family* of them. We formalize this as follows. Let $\mathcal{M}$ be a set of language models and $\mathcal{C}$ a set of conditioning contexts (prompts, domains, styles). Each pair $(M, c) \in \mathcal{M} \times \mathcal{C}$ induces a distribution $p(x \mid M, c)$ over texts. We define the **AI-generating family** as:

$$\mathcal{P}_{\text{AI}} = \left\{ p(x \mid M, c) \ : \ M \in \mathcal{M},\ c \in \mathcal{C} \right\}$$

A robust detector D must satisfy $D(x) \approx 1$ for x drawn from any $p \in \mathcal{P}_{\text{AI}}$, regardless of which model or context produced it, while satisfying $D(x) \approx 0$ for human-written text. The challenge is that $\mathcal{P}_{\text{AI}}$ is effectively infinite — the space of possible models and conditioning contexts is unbounded — so no finite training set can cover it exhaustively.

2.1 Current Approximation: Discrete Model and Domain Coverage

It is important to distinguish two roles models play in our framework. **Reference models** $\mathcal{M}_{\text{ref}}$ (Mistral-7B, Qwen-7B) are fixed instruments used to compute probabilistic features $g(x)$ from any input text — they are applied uniformly regardless of where the text came from. **Generating models** are the models whose outputs we want to detect, and it is these that define the domains in $\mathcal{P}_{\text{AI}}$. In our current setup, the detector is trained on text produced by Mistral-7B and Qwen-7B, then evaluated on text from held-out generators — Gemma-7B, Phi-3-mini, and DeepSeek-7B — that were never seen during training. Strong performance on held-out generators validates that the detector is invariant to the generating distribution, not merely to the choice of reference model.

We further approximate $\mathcal{C}$ with a discrete set of five text domains: news, web, academic, reviews, and creative writing. The detector is trained with adversarial heads for both generating model and domain, so that neither source of variation can be exploited as a shortcut. Despite this discreteness, the approximation is a principled proxy for $\mathcal{P}_{\text{AI}}$: as Figure 1 shows, our generators produce visibly distinct curvature profiles under each reference model, confirming they occupy genuinely different regions of feature space rather than clustering together.

This approach is a practical starting point but is limited: the discrete sample may miss large regions of $\mathcal{P}_{\text{AI}}$, particularly distributions induced by fine-tuned or instruction-tuned variants that lie between the models we observe.

2.2 The Infinite Size of $\mathcal{P}_{\text{AI}}$ and the Necessity of Invariance

The discrete approximation above reveals a deeper problem: $\mathcal{P}_{\text{AI}}$ is not a finite set that can ever be exhaustively sampled — it contains infinitely many distinct generating distributions, and new ones can be manufactured on demand. Fine-tuned models are the most visible example: tens of thousands of LoRA-adapted variants of open-source models exist on platforms like HuggingFace, each inducing a distinct distribution over text. But fine-tuning still produces only discrete new generators — enumerating them merely extends the training set without solving the coverage problem. A more fundamental challenge is that the space of AI-generating distributions is *continuously* infinite. We demonstrate this constructively using **activation steering vectors**: by continuously varying a coefficient $\boldsymbol{\alpha} \in \mathbb{R}^N$ applied to a base generator's hidden states, a single base model gives rise to an uncountably infinite family of distinct AI-generating distributions. No finite training set can cover this family, which means any detector that relies on memorizing the fingerprints of a fixed generator set will inevitably fail on some member of it — not because the detection problem is hard, but because the space of things to detect is infinite.

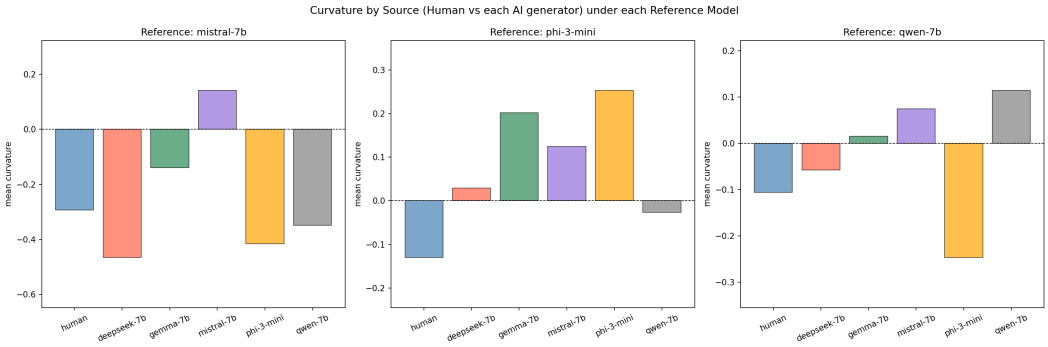

Figure 1: Mean curvature of text from each source (human and five AI generators) measured under each of the three reference models (Mistral-7B, Phi-3-mini, Qwen-7B). Each generator produces a distinct curvature profile that varies across reference models, confirming that different generators induce genuinely different distributions over probabilistic features — they are not interchangeable points in $\mathcal{P}_{\text{AI}}$. A detector trained on a subset of generators cannot assume that unseen generators will produce similar feature distributions, motivating the need for explicit invariance across the generating family.

A natural question is whether steered outputs should count as AI-generated text. The answer is unambiguous: the model weights are unchanged, and the output is still produced by an AI language model, so steered text satisfies the definition by construction. Moreover, the technique has a direct real-world analogue. LoRA fine-tuning adds low-rank updates to weight matrices that produce effects in the residual stream mathematically analogous to additive steering vectors; LoRA-adapted models are ubiquitous in practice. The continuous family $\mathcal{P}_{\text{steered}}$ is therefore a theoretically clean proxy for the much larger space of practical distribution shifts induced by adaptation, prompting, and fine-tuning — and a controlled setting in which to study whether a detector is invariant to those shifts.

Steering vector extraction. Let M be a base language model with L transformer layers. For a given layer l^*, let $h_t^{(l^*)}(x) \in \mathbb{R}^d$ denote the residual stream hidden state at token position t and $\bar{h}^{(l^*)}(x) = \frac{1}{T}\sum_{t=1}^{T} h_t^{(l^*)}(x)$ its mean over the sequence. Given AI-generated texts from two contrasting generators $\mathcal{X}_A$ and $\mathcal{X}_B$, we extract a style direction as the mean activation difference:

$$v_{AB} = \frac{1}{|\mathcal{X}_A|}\sum_{x\in\mathcal{X}_A} \bar{h}^{(l^*)}(x) \;-\; \frac{1}{|\mathcal{X}_B|}\sum_{x\in\mathcal{X}_B} \bar{h}^{(l^*)}(x)$$

We extract N such vectors $V = \{v_1, \ldots, v_N\}$ covering different generator-style axes.

Steered generation from AI text. Applying a linear combination of these vectors to an existing AI text $x \in \mathcal{X}_{\text{AI}}$ during generation shifts the producing distribution laterally within $\mathcal{P}_{\text{AI}}$:

$$\tilde{h}_t^{(l^*)} = h_t^{(l^*)} + \sum_{i=1}^{N} \alpha_i\, v_i, \qquad \boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_N) \in \mathbb{R}^N$$

Each choice of $\boldsymbol{\alpha}$ induces a distinct generating distribution $p(x \mid M, \boldsymbol{\alpha}) \in \mathcal{P}_{\text{AI}}$. Since $\boldsymbol{\alpha}$ varies continuously over $\mathbb{R}^N$, this immediately yields an uncountably infinite family:

$$\mathcal{P}_{\text{steered}} = \left\{\, p(x \mid M, \boldsymbol{\alpha}) \;:\; \boldsymbol{\alpha} \in \mathbb{R}^N \,\right\} \;\subset\; \mathcal{P}_{\text{AI}}$$

Why this makes invariant detection necessary. Because $\mathcal{P}_{\text{steered}}$ is infinite and can be instantiated from any base model at any time, a non-invariant detector can always be defeated: for any finite training set, there exist steering coefficients $\boldsymbol{\alpha}$ that place the resulting distribution outside the detector's training support, causing accuracy to degrade. Every point on the continuum is a valid AI-generating distribution, so the only principled response is to learn features that are invariant across $\mathcal{P}_{\text{AI}}$ rather than accurate on a particular finite sample of it.

This vulnerability is especially acute for detectors that score log-probability under a single fixed reference model, such as Fast-DetectGPT [1]. Steering shifts the output away from the high-probability region of the base generator, which can cause such a detector's AI-score to fall toward the human-text range even though the text is unambiguously AI-generated. By contrast, features that aggregate curvature and entropy *across* multiple reference models are more robust: no single steering direction simultaneously moves text out of the high-probability regions of all reference models, so the multi-model signal degrades more slowly. This asymmetry motivates the invariant multi-model feature design described in Section 3.

3 Defining the Invariant Probabilistic Detection Framework

3.1 Multi-Model Invariant Probabilistic Detector

Given text $x = (x_1, \ldots, x_T)$ and reference models $\mathcal{M} = \{M_1, \ldots, M_K\}$, we build a detector $D(x) \in [0, 1]$ invariant to which model generated x.

3.1.1 Probabilistic Features and Normalization

For each reference model M_k and text $x = (x_1, \ldots, x_T)$, we compute five scalar features by averaging token-level quantities over all positions $t \in \{1, \ldots, T-1\}$. Let $p_k(\cdot \mid x_{<t})$ denote the conditional distribution at position t under M_k, and let $\ell_{k,t} = \log p_k(x_t \mid x_{<t})$ be the log-probability of the actual token.

Log-likelihood measures how probable the text is under M_k:

$$\ell_k(x) = \frac{1}{T-1} \sum_{t=1}^{T-1} \ell_{k,t}$$

Token rank measures where the actual token falls in the model's ranking, as the fraction of vocabulary tokens assigned higher probability:

$$r_k(x) = \frac{1}{T-1} \sum_{t=1}^{T-1} \frac{\left|\{v \in \mathcal{V} : \log p_k(v \mid x_{<t}) > \ell_{k,t}\}\right|}{|\mathcal{V}|}$$

Margin measures the gap between the top token and the actual token in log-probability space:

$$m_k(x) = \frac{1}{T-1} \sum_{t=1}^{T-1} \Big(\max_{v \in \mathcal{V}} \log p_k(v \mid x_{<t}) - \ell_{k,t} \Big)$$

Entropy measures the uncertainty of M_k's predictive distribution at each position:

$$H_k(x) = \frac{1}{T-1} \sum_{t=1}^{T-1} \Big(- \sum_{v \in \mathcal{V}} p_k(v \mid x_{<t}) \, \log p_k(v \mid x_{<t}) \Big)$$

Curvature captures the local likelihood-maximum property of AI-generated text. Following Fast-DetectGPT [1], at each position we sample J alternative tokens $a_1, \ldots, a_J \sim p_k(\cdot \mid x_{<t})$ from the model's own distribution and measure how much the actual token's log-probability exceeds that of random draws:

$$d_k(x) = \frac{1}{T-1} \sum_{t=1}^{T-1} \left(\ell_{k,t} - \frac{1}{J} \sum_{j=1}^{J} \log p_k(a_j \mid x_{<t}) \right), \quad a_j \sim p_k(\cdot \mid x_{<t})$$

AI-generated text tends to place tokens near local maxima of the likelihood surface, so $d_k(x)$ is expected to be positive and large for AI text relative to human text.

Each feature is z-score normalized across the training corpus per model:

$$z_k^f(x) = \frac{f_k(x) - \mu_k^f}{\sigma_k^f + \varepsilon}, \quad f \in \{\ell, r, m, H, d\}$$

The 15 normalized per-model features are concatenated with two cross-model curvature statistics to form the final feature vector $g(x) \in \mathbb{R}^{17}$:

$$C_{\text{mean}}(x) = \frac{1}{K}\sum_{k=1}^{K} z_k^d(x), \qquad C_{\text{var}}(x) = \text{Var}_k\big(z_k^d(x)\big)$$

3.1.2 Invariant Representation Learning and Training

We encode $g(x)$ with a learned encoder E_θ and classify with a linear head:

$$u(x) = E_\theta(g(x)), \qquad \hat{y}(x) = \sigma(w^\top u(x) + b)$$

Training combines three objectives to enforce invariance to source model and domain:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda_1 \mathcal{L}_{\text{adv}} + \lambda_2 \mathcal{L}_{\text{ctr}}$$

where $\mathcal{L}_{\text{cls}}$ is binary cross-entropy, $\mathcal{L}_{\text{adv}} = \mathcal{L}_{\text{src}}(A_s(u(x)), s) + \mathcal{L}_{\text{dom}}(A_d(u(x)), d)$ removes source model and domain information adversarially via gradient reversal, and $\mathcal{L}_{\text{ctr}}$ is a supervised contrastive loss that pulls together representations of samples sharing the same label and pushes apart those with different labels:

$$\mathcal{L}_{\text{ctr}} = -\frac{1}{|\mathcal{P}(x)|}\sum_{x^+ \in \mathcal{P}(x)} \log \frac{\exp(\text{sim}(u(x),\, u(x^+))/\tau)}{\sum_{x' \neq x} \exp(\text{sim}(u(x),\, u(x'))/\tau)}$$

where $\mathcal{P}(x) = \{x' \neq x : y_{x'} = y_x\}$ is the set of same-label positives in the batch and τ is a temperature parameter. The final detector is $D(x) = \sigma(w^\top E_\theta(g(x)) + b)$.

4 Experimental Setup

4.1 Dataset Construction

We construct a custom dataset spanning three text domains and six generating models. Human-written text is drawn from three existing benchmarks: XSum (news summaries), WritingPrompts (open-ended creative writing), and SQuAD (Wikipedia passages). For each domain we take 100 human samples. AI-generated text is produced by prompting six open-weight models — Mistral-7B, Qwen-7B, Gemma-7B, DeepSeek-7B, Phi-3-mini, and LLaMA-2-13B — to generate completions in the same domain, with approximately 34 samples per model per domain to maintain a roughly 50/50 human-to-AI balance within each split.

4.2 Train/Test Split

A core contribution of our approach is joint invariance to both generating model and domain. To measure this, we use a strict held-out split along both axes simultaneously:

- **Training**: XSum and WritingPrompts domains; Mistral-7B, Qwen-7B, and Gemma-7B as generating models.
- **Test**: SQuAD domain; DeepSeek-7B, Phi-3-mini, and LLaMA-2-13B as generating models.

Neither the test domain nor the test generators appear during training. This means any performance on the test set cannot be attributed to memorizing the style of a known generator or the statistical regularities of a known domain — it must come from genuinely invariant features.

4.3 Probabilistic Feature Extraction

For each text we compute the 17-dimensional feature vector $g(x) \in \mathbb{R}^{17}$ described in Section 3, using three fixed reference models (Mistral-7B, Phi-3-mini, Qwen-7B) applied uniformly to all samples regardless of their origin. Features are z-score normalized per feature per reference model using statistics computed on the training split only; test samples are normalized with the same training statistics to avoid leakage.

4.4 Model Training

With the feature vectors and labels in hand, we instantiate and train the InvariantDetector from Section 3. The encoder $E_\theta : \mathbb{R}^{17} \to \mathbb{R}^{32}$ is a two-layer MLP (17→64→32, ReLU activations), and the classifier is a single linear layer. Each training sample carries three labels: binary AI/human (y), generating model identity (s), and domain (d). These feed directly into the three objectives from Section 3.2 with $\lambda_1 = \lambda_2 = 0.1$: $\mathcal{L}_{\text{cls}}$ supervises detection, $\mathcal{L}_{\text{adv}}$ applies gradient reversal through adversarial heads for s and d to strip source and domain information from $u(x)$, and $\mathcal{L}_{\text{ctr}}$ is a supervised contrastive loss that pulls same-label representations together and pushes apart opposite-label representations in the embedding space. The adversarial heads see all six generating models and all three domains during training, forcing the representation to be flat across the full range of variation in the training split — making it more likely to remain flat on the held-out generators and domain at test time.

5 Evaluation

We evaluate across two complementary settings: a held-out invariance test on our own dataset, and a direct comparison on the Pangram benchmark.

5.1 Invariance Evaluation on Custom Dataset

Dataset and split. Our dataset consists of three domains — XSum (news), WritingPrompts (creative), and SQuAD (Wikipedia/academic) — and six generating models: Mistral-7B, Qwen-7B, DeepSeek-7B, Gemma-7B, Phi-3-mini, and LLaMA-3-8B. We use a strict held-out split along *both* axes simultaneously:

- **Training**: XSum and WritingPrompts domains, Mistral-7B and Qwen-7B generators.
- **Test**: SQuAD domain, DeepSeek-7B generator.

The test set is therefore doubly out-of-distribution — the domain and the generator are both unseen during training. Strong test performance validates that the detector is jointly invariant to domain shift and distributional shift in the generating model, not merely one or the other.

Baselines. We compare our InvariantDetector against the **Pangram EditLens** model [2] — a RoBERTa-large supervised detector fine-tuned on the Pangram training set, representing the state-of-the-art in-distribution detector for this benchmark. Both models are evaluated without retraining or fine-tuning on the doubly out-of-distribution test set, isolating the effect of invariant training rather than data advantage.

Metrics. We report AUROC as the primary threshold-free metric, along with accuracy and F1 at the 0.5 decision threshold.

5.2 Benchmark Comparison on Pangram Dataset

To situate our approach in the broader detection literature, we train our InvariantDetector on the Pangram training set and evaluate it on the Pangram test set alongside the Pangram model and Fast-DetectGPT. We note that the Pangram benchmark is an in-distribution evaluation: the test set is drawn from the same domain and generator mixture as the training set, so strong performance reflects memorization of the training distribution rather than generalization to unseen sources. This makes it a favorable setting for the Pangram model and an upper bound on what any detector trained on that data can achieve. Our goal in this comparison is not to claim superiority on a benchmark designed for in-distribution evaluation, but to verify that our invariance objectives and probabilistic features do not sacrifice in-distribution accuracy — and to contrast this setting explicitly with the doubly out-of-distribution evaluation in §4.1, where in-distribution methods are expected to degrade.

Results. Figures 2 and 3 report AUROC across the two evaluation settings; together they reveal a clear asymmetry between the two approaches.

Experiment 1 — doubly OOD ($n = 186$). Our InvariantDetector achieves an AUROC of 0.914. EditLens collapses to 0.699 — barely above chance — and its F1 falls to 0.286 versus 0.783 for our model. EditLens memorises the Pangram training distribution and cannot transfer to unseen domains or generators.

Experiment 2 — Pangram in-distribution ($n = 4081$). EditLens recovers to a near-perfect 0.9997, confirming it is highly specialised to its training distribution. Our model scores 0.981, remaining competitive even in a setting that is by design most favourable to EditLens.

The two experiments isolate the core failure mode of distribution-specific supervised detectors: strong in-distribution performance that collapses under domain and generator shift. Our invariant detector avoids this collapse while retaining competitive in-distribution accuracy, demonstrating that invariant training over probabilistic multi-model features generalises where specialised supervised methods do not.

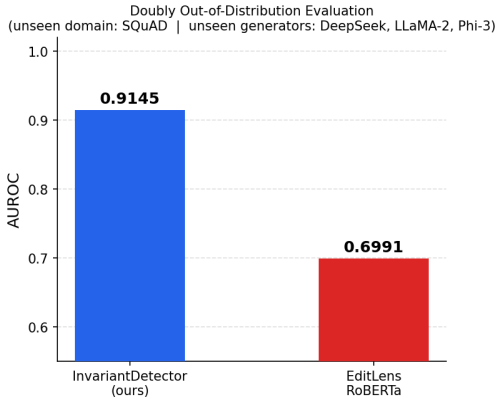

Figure 2: AUROC on the doubly out-of-distribution test set (unseen domain: SQuAD; unseen generators: DeepSeek-7B, LLaMA-2-13B, Phi-3-mini). EditLens degrades sharply while our InvariantDetector maintains strong performance.

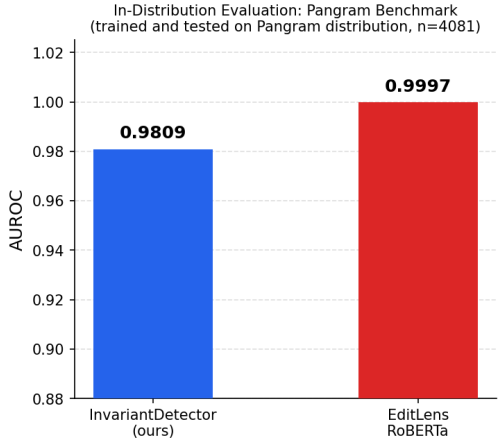

Figure 3: AUROC on the Pangram in-distribution benchmark ($n = 4081$). EditLens dominates in its native setting; our model remains within two points despite being trained with invariance constraints rather than optimised for this distribution.

6 Conclusion

We have argued that the core difficulty in AI-generated text detection is not the classification problem itself, but the unbounded size of $\mathcal{P}_{\text{AI}}$: the family of AI-generating distributions is infinite, continuously extensible via steering, and cannot be exhaustively covered by any finite training set. This motivates building detectors that are invariant to which generator and domain produced a text, rather than detectors that memorise the fingerprints of a fixed set of generators.

Our approach extracts a 17-dimensional feature vector from three reference language models — capturing log-likelihood, token rank, margin, entropy, and curvature — and trains a small MLP with adversarial domain/generator heads and a supervised contrastive loss to suppress generator- and domain-specific signal. The result is a lightweight detector that generalises across distribution shifts without sacrificing in-distribution accuracy.

Empirically, this plays out as predicted. On a doubly out-of-distribution test set (unseen domain and unseen generators simultaneously), our InvariantDetector achieves an AUROC of 0.914 while EditLens — the state-of-the-art supervised detector trained on the Pangram benchmark — collapses to 0.699. On the Pangram in-distribution benchmark, EditLens recovers to near-perfect 0.9997 while our model scores 0.981, confirming that invariant training does not sacrifice in-distribution performance. The two experiments together expose the central failure mode of distribution-specific supervised detectors: strong accuracy within their training distribution that degrades sharply outside it.

A natural direction for future work is to empirically validate the theoretical claim of Section 2.2 by applying activation steering vectors to AI-generated text and measuring detector degradation as a function of the steering coefficient α. We expect that detectors relying on a single reference model's log-probability will degrade faster under steering than our multi-model invariant features, which would provide a direct empirical proof that invariant detection is necessary as $\mathcal{P}_{\text{AI}}$ expands. This will hopefully be done by the final draft.

References

[1] Guangsheng Bao, Yanbin Zhao, Zhiyang Teng, Linyi Yang, and Yue Zhang. Fast-DetectGPT: Efficient zero-shot detection of machine-generated text via conditional probability curvature. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.05130.

[2] Katherine Thai, Bradley Emi, Elyas Masrour, and Mohit Iyyer. EditLens: Quantifying the extent of AI editing in text. *arXiv preprint arXiv:2510.03154*, 2025.